INFORME DE LABORATORIO

INFORMACIÓN BÁSICA

ASIGNATURA:	Sistemas Distribuidos				
TÍTULO DE LA PRÁCTICA:	Sockets				
NÚMERO DE LA PRÁCTICA:	03	**AÑO LECTIVO:**	2026	**NRO. SEMESTRE:**	2026-A
FECHA DE PRESENTACIÓN:	29/04/2026	**HORA DE PRESENTACIÓN:**	11:59:00		
INTEGRANTE(s): - Bedregal Perez Daniel - Jara Mamani Mariel Alisson - Mestas Zegarra Christian Raul - Noa Camino Yenaro Joel - Sequeiros Condori Luis Gustavo				**NOTA (0 - 20):**	Nota colocada por el docente
DOCENTE: Mg. Maribel Molina Barriga					

RESULTADOS Y PRUEBAS

ENLACE A GITHUB

https://github.com/christianmz565/sd-2026-lab-c-grupo-3/tree/main/l3

SOLUCIÓN DE EJERCICIOS RESUELTOS

Ejemplo de Sockets en Java: Programa Servidor

El archivo `Servidor.java` define la clase principal que se encarga de escuchar peticiones en el puerto `5000` mediante un objeto de la clase `ServerSocket`.

La lógica principal se encuentra en su constructor, donde entra en un bucle acotado a tres iteraciones para aceptar hasta tres conexiones de clientes utilizando la primitiva `accept()`.

Para cada conexión establecida, el servidor recupera el flujo de salida a través de `getOutputStream()` y lo envuelve en un `DataOutputStream` para poder escribir cadenas de texto formateadas.

Posteriormente, el servidor transmite un mensaje de saludo personalizado utilizando el método `writeUTF(...)` y cierra la conexión actual con `close()` para liberar los recursos del socket.

```java
import java.io.*;
import java.net.*;

public class Servidor {
    static final int PUERTO = 5000;

    public Servidor() {
        try {
            ServerSocket skServidor = new ServerSocket(PUERTO);
            System.out.println("Escucho el puerto " + PUERTO);
            for (int numCli = 0; numCli < 3; numCli++) {
                Socket skCliente = skServidor.accept(); // Crea objeto
                System.out.println("Sirvo al cliente " + numCli);
                OutputStream aux = skCliente.getOutputStream();
                DataOutputStream flujo = new DataOutputStream(aux);
                flujo.writeUTF("Hola cliente " + numCli);
                skCliente.close();
```

```
            }
            System.out.println("Demasiados clientes por hoy");
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }

    public static void main(String[] arg) {
        new Servidor();
    }
}
```

Ejemplo de Sockets en Java: Programa Cliente

El archivo `Cliente.java` define la aplicación cliente que busca conectarse al servidor especificado en la constante `HOST` (localhost) a través del puerto `5000`.

En su ejecución, instancia un objeto `Socket` que intenta establecer la conexión de red de forma síncrona.

Una vez conectado, el cliente obtiene el flujo de entrada con `getInputStream()` y lo envuelve en un `DataInputStream` para leer el flujo binario como una cadena de caracteres UTF-8.

Finalmente, el mensaje recibido se imprime en la consola estándar del sistema y la conexión se termina explícitamente llamando al método `close()` del socket.

```
import java.io.*;
import java.net.*;

public class Cliente {
    static final String HOST = "localhost";
    static final int PUERTO = 5000;

    public Cliente() {
        try {
            Socket skCliente = new Socket(HOST, PUERTO);
            InputStream aux = skCliente.getInputStream();
            DataInputStream flujo = new DataInputStream(aux);
            System.out.println(flujo.readUTF());
            skCliente.close();
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }

    public static void main(String[] arg) {
        new Cliente();
    }
}
```

A continuación, se muestra el resultado de compilar y ejecutar el servidor y los clientes.

El servidor escucha el puerto y atiende secuencialmente a los clientes, los cuales imprimen el mensaje de saludo recibido.

```
❯ javac Servidor.java
❯ java Servidor
Escucho el puerto 5000
Sirvo al cliente 0
Sirvo al cliente 1
Sirvo al cliente 2
Demasiados clientes por hoy
```

```
❯ javac Cliente.java
❯ java Cliente
Hola cliente 0
❯ java Cliente
Hola cliente 1
❯ java Cliente
Hola cliente 2
```

SOLUCIÓN DE EJERCICIOS PROPUESTOS

Sistema de Chat en Java: Clases de Mensaje

El archivo `ChatMessage.java` implementa la clase serializable que define la estructura y el contenido de los mensajes intercambiados entre los clientes y el servidor.

Esta clase permite encapsular tanto el tipo de comando (como `WHOISIN`, `MESSAGE` o `LOGOUT`) como el contenido en texto plano.

Al implementar la interfaz `Serializable`, los objetos de esta clase pueden ser transmitidos directamente a través de la red utilizando flujos de objetos, lo que evita la necesidad de realizar procesamiento y análisis manual de bytes en los extremos de la comunicación.

```java
import java.io.*;

/*
 * This class defines the different type of messages that will be exchanged between the
 * Clients and the Server.
 * When talking from a Java Client to a Java Server a lot easier to pass Java objects, no
 * need to count bytes or to wait for a line feed at the end of the frame
 */
public class ChatMessage implements Serializable {
    static final int WHOISIN = 0, MESSAGE = 1, LOGOUT = 2;
    private int type;
    private String message;

    ChatMessage(int type, String message) {
        this.type = type;
        this.message = message;
    }

    int getType() { return type; }
    String getMessage() { return message; }
}
```

Sistema de Chat en Java: Programa Servidor

El archivo `Server.java` implementa el nodo central del sistema de chat, responsable de coordinar las conexiones y retransmitir los mensajes a los usuarios activos.

En el método `start()`, el servidor inicializa un `ServerSocket` e ingresa en un ciclo continuo donde invoca `accept()` para recibir nuevas peticiones de conexión.

Cada conexión nueva se delega a un hilo independiente mediante la clase interna `ClientThread`, lo que permite al servidor manejar múltiples clientes de forma concurrente sin bloquearse.

La comunicación se gestiona mediante métodos sincronizados como `broadcast(...)`, el cual itera sobre la lista de clientes activos y utiliza `ObjectOutputStream` para transmitir los mensajes serializados, soportando además el envío de mensajes privados mediante la sintaxis `@usuario`.

```java
import java.io.*;
import java.net.*;
import java.text.SimpleDateFormat;
import java.util.*;

public class Server {
    private static int uniqueId;
    private ArrayList<ClientThread> al;
```

```java
private SimpleDateFormat sdf;
private int port;
private boolean keepGoing;
private String notif = " *** ";

public Server(int port) {
    this.port = port;
    sdf = new SimpleDateFormat("HH:mm:ss");
    al = new ArrayList<ClientThread>();
}

public void start() {
    keepGoing = true;
    try {
        ServerSocket serverSocket = new ServerSocket(port);
        while (keepGoing) {
            display("Server waiting for Clients on port " + port + ".");
            Socket socket = serverSocket.accept();
            if (!keepGoing) break;
            ClientThread t = new ClientThread(socket);
            al.add(t);
            t.start();
        }
        try {
            serverSocket.close();
            for (int i = 0; i < al.size(); ++i) {
                ClientThread tc = al.get(i);
                try {
                    tc.sInput.close();
                    tc.sOutput.close();
                    tc.socket.close();
                } catch (IOException ioE) {}
            }
        } catch (Exception e) {
            display("Exception closing the server and clients: " + e);
        }
    } catch (IOException e) {
        String msg = sdf.format(new Date()) + " Exception on new ServerSocket: " + e + "\n";
        display(msg);
    }
}

protected void stop() {
    keepGoing = false;
    try {
        new Socket("localhost", port);
    } catch (Exception e) {}
}

private void display(String msg) {
    String time = sdf.format(new Date()) + " " + msg;
    System.out.println(time);
}

private synchronized boolean broadcast(String message) {
    String time = sdf.format(new Date());
    String[] w = message.split(" ", 3);
    boolean isPrivate = false;
    if (w.length > 1 && w[1].charAt(0) == '@') isPrivate = true;

    if (isPrivate) {
        String tocheck = w[1].substring(1, w[1].length());
        message = w[0] + w[2];
        String messageLf = time + " " + message + "\n";
        boolean found = false;
        for (int y = al.size(); --y >= 0;) {
            ClientThread ct1 = al.get(y);
            String check = ct1.getUsername();
            if (check.equals(tocheck)) {
                if (!ct1.writeMsg(messageLf)) {
                    al.remove(y);
                    display("Disconnected Client " + ct1.username + " removed from list.");
```

```
                }
                found = true;
                break;
            }
        }
        if (!found) return false;
    } else {
        String messageLf = time + " " + message + "\n";
        System.out.print(messageLf);
        for (int i = al.size(); --i >= 0;) {
            ClientThread ct = al.get(i);
            if (!ct.writeMsg(messageLf)) {
                al.remove(i);
                display("Disconnected Client " + ct.username + " removed from list.");
            }
        }
    }
    return true;
}

synchronized void remove(int id) {
    String disconnectedClient = "";
    for (int i = 0; i < al.size(); ++i) {
        ClientThread ct = al.get(i);
        if (ct.id == id) {
            disconnectedClient = ct.getUsername();
            al.remove(i);
            break;
        }
    }
    broadcast(notif + disconnectedClient + " has left the chat room." + notif);
}

public static void main(String[] args) {
    int portNumber = 1500;
    switch (args.length) {
        case 1:
            try {
                portNumber = Integer.parseInt(args[0]);
            } catch (Exception e) {
                System.out.println("Invalid port number.");
                System.out.println("Usage is: > java Server [portNumber]");
                return;
            }
        case 0:
            break;
        default:
            System.out.println("Usage is: > java Server [portNumber]");
            return;
    }
    Server server = new Server(portNumber);
    server.start();
}

class ClientThread extends Thread {
    Socket socket;
    ObjectInputStream sInput;
    ObjectOutputStream sOutput;
    int id;
    String username;
    ChatMessage cm;
    String date;

    ClientThread(Socket socket) {
        id = ++uniqueId;
        this.socket = socket;
        System.out.println("Thread trying to create Object Input/Output Streams");
        try {
            sOutput = new ObjectOutputStream(socket.getOutputStream());
            sInput = new ObjectInputStream(socket.getInputStream());
            username = (String) sInput.readObject();
            broadcast(notif + username + " has joined the chat room." + notif);
```

```java
            } catch (IOException e) {
                display("Exception creating new Input/output Streams: " + e);
                return;
            } catch (ClassNotFoundException e) {}
            date = new Date().toString() + "\n";
        }

        public String getUsername() { return username; }
        public void setUsername(String username) { this.username = username; }

        public void run() {
            boolean keepGoing = true;
            while (keepGoing) {
                try {
                    cm = (ChatMessage) sInput.readObject();
                } catch (IOException e) {
                    display(username + " Exception reading Streams: " + e);
                    break;
                } catch (ClassNotFoundException e2) {
                    break;
                }
                String message = cm.getMessage();
                switch (cm.getType()) {
                    case ChatMessage.MESSAGE:
                        boolean confirmation = broadcast(username + ": " + message);
                        if (!confirmation) {
                            String msg = notif + "Sorry. No such user exists." + notif;
                            writeMsg(msg);
                        }
                        break;
                    case ChatMessage.LOGOUT:
                        display(username + " disconnected with a LOGOUT message.");
                        keepGoing = false;
                        break;
                    case ChatMessage.WHOISIN:
                        writeMsg("List of the users connected at " + sdf.format(new Date()) + "\n");
                        for (int i = 0; i < al.size(); ++i) {
                            ClientThread ct = al.get(i);
                            writeMsg((i + 1) + ") " + ct.username + " since " + ct.date);
                        }
                        break;
                }
            }
            remove(id);
            close();
        }

        private void close() {
            try { if (sOutput != null) sOutput.close(); } catch (Exception e) {}
            try { if (sInput != null) sInput.close(); } catch (Exception e) {}
            try { if (socket != null) socket.close(); } catch (Exception e) {}
        }

        private boolean writeMsg(String msg) {
            if (!socket.isConnected()) {
                close();
                return false;
            }
            try {
                sOutput.writeObject(msg);
            } catch (IOException e) {
                display(notif + "Error sending message to " + username + notif);
                display(e.toString());
            }
            return true;
        }
    }
}
```

Sistema de Chat en Java: Programa Cliente

El archivo `Client.java` contiene la lógica del cliente, el cual establece la conexión bidireccional y maneja la interacción con el usuario final.

Durante su inicialización, el cliente solicita un nombre de usuario y se conecta al servidor, configurando inmediatamente los flujos de objetos `ObjectInputStream` y `ObjectOutputStream`.

Para evitar bloquear la entrada del usuario mientras se esperan nuevos mensajes, el cliente lanza un hilo secundario `ListenFromServer` que lee continuamente del flujo de entrada y muestra los mensajes recibidos en pantalla.

Al mismo tiempo, el hilo principal procesa comandos y cadenas de texto ingresadas en la consola, construyendo objetos `ChatMessage` y enviándolos al servidor para su distribución.

```java
import java.net.*;
import java.io.*;
import java.util.*;

public class Client {
    private String notif = " *** ";
    private ObjectInputStream sInput;
    private ObjectOutputStream sOutput;
    private Socket socket;
    private String server, username;
    private int port;

    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }

    Client(String server, int port, String username) {
        this.server = server;
        this.port = port;
        this.username = username;
    }

    public boolean start() {
        try {
            socket = new Socket(server, port);
        } catch (Exception ec) {
            display("Error connecting to server:" + ec);
            return false;
        }

        String msg = "Connection accepted " + socket.getInetAddress() + ":" + socket.getPort();
        display(msg);

        try {
            sInput = new ObjectInputStream(socket.getInputStream());
            sOutput = new ObjectOutputStream(socket.getOutputStream());
        } catch (IOException eIO) {
            display("Exception creating new Input/output Streams: " + eIO);
            return false;
        }

        new ListenFromServer().start();

        try {
            sOutput.writeObject(username);
        } catch (IOException eIO) {
            display("Exception doing login : " + eIO);
            disconnect();
            return false;
        }
        return true;
    }

    private void display(String msg) {
        System.out.println(msg);
    }

    void sendMessage(ChatMessage msg) {
```

```
        try {
            sOutput.writeObject(msg);
        } catch (IOException e) {
            display("Exception writing to server: " + e);
        }
    }

    private void disconnect() {
        try { if (sInput != null) sInput.close(); } catch (Exception e) {}
        try { if (sOutput != null) sOutput.close(); } catch (Exception e) {}
        try { if (socket != null) socket.close(); } catch (Exception e) {}
    }

    public static void main(String[] args) {
        int portNumber = 1500;
        String serverAddress = "localhost";
        String userName = "Anonymous";
        Scanner scan = new Scanner(System.in);
        System.out.println("Enter the username: ");
        userName = scan.nextLine();

        switch (args.length) {
            case 3:
                serverAddress = args[2];
            case 2:
                try {
                    portNumber = Integer.parseInt(args[1]);
                } catch (Exception e) {
                    System.out.println("Invalid port number.");
                    System.out.println("Usage is: > java Client [username] [portNumber] [serverAddress]");
                    return;
                }
            case 1:
                userName = args[0];
            case 0:
                break;
            default:
                System.out.println("Usage is: > java Client [username] [portNumber] [serverAddress]");
                return;
        }

        Client client = new Client(serverAddress, portNumber, userName);
        if (!client.start()) return;

        System.out.println("\nHello.! Welcome to the chatroom.");
        System.out.println("Instructions:");
        System.out.println("1. Simply type the message to send broadcast to all active clients");
        System.out.println("2. Type '@username<space>yourmessage' without quotes to send message to desired client");
        System.out.println("3. Type 'WHOISIN' without quotes to see list of active clients");
        System.out.println("4. Type 'LOGOUT' without quotes to logoff from server");

        while (true) {
            System.out.print("> ");
            String msg = scan.nextLine();
            if (msg.equalsIgnoreCase("LOGOUT")) {
                client.sendMessage(new ChatMessage(ChatMessage.LOGOUT, ""));
                break;
            } else if (msg.equalsIgnoreCase("WHOISIN")) {
                client.sendMessage(new ChatMessage(ChatMessage.WHOISIN, ""));
            } else {
                client.sendMessage(new ChatMessage(ChatMessage.MESSAGE, msg));
            }
        }
        scan.close();
        client.disconnect();
    }

    class ListenFromServer extends Thread {
        public void run() {
            while (true) {
                try {
                    String msg = (String) sInput.readObject();
```

```
                    System.out.println(msg);
                    System.out.print("> ");
                } catch (IOException e) {
                    display(notif + "Server has closed the connection: " + e + notif);
                    break;
                } catch (ClassNotFoundException e2) { }
            }
        }
    }
}
```

A continuación, se muestra el resultado de compilar y ejecutar el sistema de chat propuesto.

El servidor administra las conexiones concurrentemente mientras que los clientes interactúan mediante comandos de sistema y mensajes públicos y privados.

```
❯ javac *.java
❯ java Server
22:47:50 Server waiting for Clients on port 1500.
Thread trying to create Object Input/Output Streams
22:47:50  *** Bob has joined the chat room. ***
22:47:50 Server waiting for Clients on port 1500.
Thread trying to create Object Input/Output Streams
22:47:51  *** Alice has joined the chat room. ***
22:47:51 Server waiting for Clients on port 1500.
22:47:52 Alice: Hello everyone
22:47:53 Bob disconnected with a LOGOUT message.
22:47:53  *** Bob has left the chat room. ***
22:47:54 Alice disconnected with a LOGOUT message.
22:47:54  *** Alice has left the chat room. ***
```

```
❯ java Client
Alice
Enter the username:
Connection accepted localhost/127.0.0.1:1500

Hello.! Welcome to the chatroom.
Instructions:
1. Simply type the message to send broadcast to all active clients
2. Type '@username<space>yourmessage' without quotes to send message to desired client
3. Type 'WHOISIN' without quotes to see list of active clients
4. Type 'LOGOUT' without quotes to logoff from server
> WHOISIN
22:47:51 Bob:Hi Alice

> > List of the users connected at 22:47:51

> 1) Bob since Wed Apr 29 22:47:50 PET 2026

> 2) Alice since Wed Apr 29 22:47:51 PET 2026

> Hello everyone
> 22:47:52 Alice: Hello everyone

> 22:47:53  *** Bob has left the chat room. ***

> LOGOUT
 *** Server has closed the connection: java.net.SocketException: Socket closed ***
```

```
❯ java Client
Bob
Enter the username:
Connection accepted localhost/127.0.0.1:1500

Hello.! Welcome to the chatroom.
Instructions:
1. Simply type the message to send broadcast to all active clients
2. Type '@username<space>yourmessage' without quotes to send message to desired client
3. Type 'WHOISIN' without quotes to see list of active clients
4. Type 'LOGOUT' without quotes to logoff from server
> WHOISIN
> List of the users connected at 22:47:51

> 1) Bob since Wed Apr 29 22:47:50 PET 2026

> 22:47:51  *** Alice has joined the chat room. ***

> @Alice Hi Alice
> 22:47:52 Alice: Hello everyone

> LOGOUT
 *** Server has closed the connection: java.net.SocketException: Socket closed ***
```

CUESTIONARIO

¿Por qué y en qué momento usar sockets TCP y sockets UDP?

Los sockets TCP se utilizan cuando se requiere una comunicación confiable, en orden y sin pérdida de datos entre los extremos, ya que el protocolo implementa mecanismos de retransmisión y control de congestión.

Se deben elegir en aplicaciones donde la integridad de la información es crítica, como en el caso de la transferencia de archivos (FTP), la web (HTTP) o servicios de acceso remoto (Telnet).

Por otro lado, los sockets UDP se utilizan cuando la prioridad es la velocidad y el bajo costo computacional, a expensas de la confiabilidad, ya que los paquetes se envían de forma independiente y sin garantía de entrega.

Su uso es adecuado en escenarios donde la pérdida ocasional de un datagrama es tolerable y preferible a un retraso acumulado, como en transmisiones de video en vivo, comunicaciones de voz sobre IP (VoIP) o juegos en línea de ritmo rápido.

En un socket TCP, ¿cuándo sabe el servidor que el cliente ha cerrado la conexión?

El servidor detecta el cierre de la conexión de manera asíncrona cuando intenta interactuar con el flujo de datos asociado al socket del cliente.

Si el cliente realiza un cierre ordenado (graceful shutdown), las operaciones de lectura en el servidor, como `read()` o `readObject()`, devolverán valores de terminación (como `-1` o `null`) o lanzarán una excepción de fin de archivo como `EOFException`.

Si el cierre es abrupto, como en el caso de una pérdida de conexión de red o cierre forzado del proceso, el sistema operativo subyacente lo detectará durante los intentos de lectura o escritura y la máquina virtual de Java lanzará una excepción del tipo `SocketException` o `IOException`, indicando que la conexión ha sido reiniciada (connection reset).

CONCLUSIONES Y RECOMENDACIONES

CONCLUSIONES

1. La ejecución de la prueba inicial verificó que la clase `ServerSocket` facilita la creación de servicios en red mediante primitivas bloqueantes que abstraen la complejidad subyacente del protocolo TCP.
2. La implementación del sistema de chat demostró que el uso de hilos independientes (como `ClientThread`) es esencial para manejar clientes concurrentes sin afectar el ciclo principal de aceptación de conexiones del servidor.

3. Se observó que el intercambio de objetos serializables en Java a través de `ObjectInputStream` y `ObjectOutputStream` simplifica significativamente la programación de protocolos de aplicación al eliminar la necesidad de parsear cadenas de bytes manualmente.

RECOMENDACIONES

1. Integrar mecanismos robustos de manejo de latidos (heartbeats) para identificar y desconectar proactivamente a clientes caídos sin depender únicamente de excepciones de entrada y salida.
2. Considerar la migración de la gestión de hilos a las abstracciones de concurrencia modernas de Java o utilizar el paquete NIO (Non-blocking I/O) para mejorar la escalabilidad frente a un alto volumen de clientes simultáneos.
3. Extender el sistema para incorporar validación de esquemas y cifrado de los objetos serializables en tránsito, previniendo posibles ataques de inyección y escuchas no autorizadas en redes públicas.

REFERENCIAS Y BIBLIOGRAFÍA

[1] Tanenbaum, A.S. (2008). Sistemas distribuidos: principios y paradigmas. México. Pearson Educación.

[2] Ceballos, F. J. (2006). Java 2, Curso de programación. México: Alfaomega, RaMa.

[3] Deitel, H. M., & Deitel, P. J. (2004). Cómo programar en Java. México: Pearson Educación.

[4] García Tomás, J., Ferrando, S., & Piattini, M. (2001). Redes para procesos distribuidos. México: Alfaomega Ra-Ma.

[5] Orfali, R., & Harkey, D. (1998). Client/Server Programming with Java and CORBA. USA: Wiley